

Laboratory 5

Creați operații CRUD încapsulate în proceduri stocate pentru tabelele implicate într-o relație **M:N** din baza de date creată în prima temă de laborator, astfel încât fiecărui tabel să-i corespundă câte 4 proceduri stocate (una pentru fiecare operație CRUD).

Trebuie să folosiți:

- parametri de intrare/ieșire;
- funcții (de exemplu pentru formatarea/validarea datelor de intrare);
- constrângeri pe tabelă/coloană pentru a asigura validitatea datelor.

CRUD Operations

Create = Create new table or INSERT

Read = SELECT

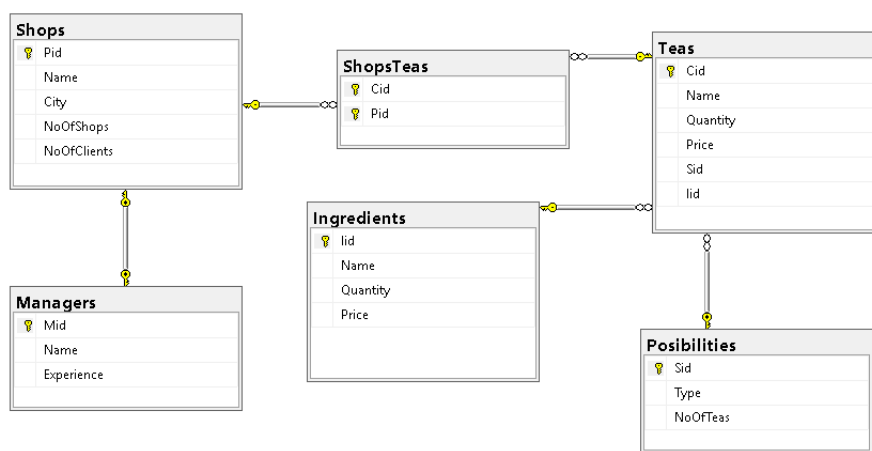
Update = UPDATE

Delete = DELETE

You have to create stored procedures for CRUD operations for 3 tables that are in the **M:N** relationship (4 operations*3 tables = 12 stored procedures).

Optional – one extra stored procedure CRUD per table that will incapsulate the 4 corresponding operations.

You will have to consider your own database and work on it. Here, we consider the database



We can choose tables Shops – (ShopsTeas) – Teas – relationship **M-N**.

The way that you implement your **stored procedures** is up to you!!!

An example of crud operations on a table can be (similar for Shops, Ingredients, Possibilities – these tables have only primary keys, no foreign keys)

```
USE [Example_Lab1]
GO
CREATE PROCEDURE [dbo].[INSERT_Shops]
@table_name Varchar(50), @name varchar(50), @city varchar(50), @nos int, @noc int, @noOfRows int
AS
BEGIN
    SET NOCOUNT ON;
    -- verify the parameters - at least one from the list - with the help of a scalar function or a stored
    procedure with output parameter
    -- CREATE=INSERT
```

```

declare @n int =1
-- we add as many rows as the parameter indicate us
-- not all the fields must be given as parameters
while @n<=@noOfRows begin
    insert into Shops(Name, City, NoOfShops, NoOfClients)
        Values(@name, @city, @nos, @noc)
    set @n=@n+1
end
print 'INSERT operation for table ' + @table_name
END

CREATE PROCEDURE [dbo].[SELECT_Shops] @table_name Varchar(50)
AS
BEGIN
    -- READ=SELECT
    select * from Shops
    print 'SELECT operation for table ' + @table_name
END

CREATE PROCEDURE [dbo].[UPDATE_Shops]
@table_name Varchar(50), @name varchar(50), @city varchar(50), @nos int, @noc int
AS
BEGIN
    SET NOCOUNT ON;
    -- verify the parameters - at least one from the list – to be valid to update
    -- UPDATE
    update Shops
    set Name=@name, City=@city, NoOfShops=@nos, NoOfClients=@noc
    where NoOfClients>10 and Name LIKE 'A%'
    print 'UPDATE operation for table ' + @table_name
END

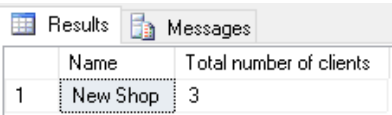
CREATE PROCEDURE [dbo].[DELETE_Shops]
@table_name Varchar(50), @name varchar(50), @city varchar(50), @nos int, @noc int
AS
BEGIN
    -- VALIDATE THE DELETE +/- parameters
    -- if does not crash
    -- DELETE
    delete from Shops
    where Name=@name OR City=@city OR NoOfShops=@nos OR NoOfClients=@noc
    -- otherwise PRINT a message 'DELETE is not possible to be handled'
    print 'DELETE operation for table ' + @table_name
END

```

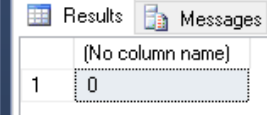
For the instruction SELECT, one can also use a **function** that returns a table, or a view, or a stored procedure.

- *inline/multi Table-Valued Functions – RETURNS a TABLE*

USE Example_Lab1 GO	-- EXECUTE SELECT * FROM ShopsF (2);
------------------------	---

<pre> IF OBJECT_ID (N'ShopsF', N'IF') IS NOT NULL DROP FUNCTION ShopsF; GO CREATE FUNCTION ShopsF (@Pid int) RETURNS TABLE AS RETURN (SELECT Name, SUM(NoOfClients) AS 'Total number of clients' FROM Shops WHERE Pid=@Pid GROUP BY Name); GO </pre>	 Return the Name and the Number of Clients for a Shop given with ID
--	---

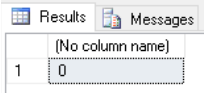
- *Scalar-Valued Functions* - used to verify the parameters

<pre> Use Example_Lab1 IF OBJECT_ID (N'dbo.Sfunction', N'FN') IS NOT NULL DROP FUNCTION dbo.Sfunction; GO -- return the Total number of clients for a given Shops Pid CREATE FUNCTION dbo.Sfunction(@Pid int) RETURNS int AS BEGIN DECLARE @r int; SELECT @r = SUM(NoOfClients) FROM Shops WHERE Pid = @Pid AND Name LIKE 'S%'; IF (@r IS NULL) SET @r = 0; RETURN @r; END; GO </pre>	<pre>-- EXECUTION</pre> <pre>SELECT dbo.Sfunction(2)</pre> 
---	--

One **must create at least one function for each table** in which will verify some conditions related to a field of the table (that will appear as a parameter in the crud stored procedure)

For example: verify that the name of the Shop start with a letter, verify that the quantity for Ingredient to be positive, verify the e-mail address to respect a format, ...

TestPrice – check for table Ingredients the field Price - when will be executed

<pre> Create function dbo.TestPrice(@p int) RETURNS INT AS BEGIN IF @p BETWEEN 10 AND 20 SET @p=1 ELSE SET @p=0 RETURN @p END </pre>	<pre>-- execute</pre> <pre>SELECT dbo.TestPrice(2)</pre> 	<pre>--or verification like</pre> <pre>if dbo.TestPrice(@p)=1</pre> <pre> insert into...</pre> <pre>else print 'no insertion..'</pre>
--	--	--

The validation functions can check a variable to be in a specified interval, a varchar to have a specified length, a specified varchar to have just letters, a date to be smaller/greater than the current date (e.g. expiration_date>GETDATE()), the e-mail address to include the character '@', and so on...

Function that add a constraint with check (**constrângeri pe tabelă/coloană pentru a asigura validitatea datelor** – primary key, foreign key, unique, check, default, ...)

```
CREATE TABLE CheckTbl (col1 int, col2 int);
GO
CREATE FUNCTION CheckFnctn()
RETURNS int
AS
BEGIN
    DECLARE @retval int
    SELECT @retval = COUNT(*) FROM CheckTbl
    RETURN @retval
END;
GO
ALTER TABLE CheckTbl
ADD CONSTRAINT chkRowCount CHECK (dbo.CheckFnctn() >= 1 );
GO
```

Stored Procedure with INPUT/OUTPUT parameters

```
CREATE PROCEDURE test_InsertShops
@flag bit OUTPUT,-- return 0 for fail,1 for success
@Name varchar(50),
@City varchar(100),
@NoOfShops int,
@NoOfClients int
AS
BEGIN
    Insert into Shops(Name, City, NoOfShops, NoOfClients) Values(@Name, @City, @NoOfShops,
@NoOfClients)
    IF @@TRANCOUNT > 0 SET @flag=1;
    ELSE SET @flag=0;
END
```

```
--Execute above created procedure to insert rows into table
Declare @flag bit
EXEC test_InsertShops @flag OUTPUT,'Shop 1','Bucuresti', 14, 12
if @flag=1 print 'Successfully inserted'
else print 'There is some error'
```

(1 row(s) affected)
There is some error

The parameters of the stored procedures must be parameters that refers to the columns from the table considered or / and also others.

Insert - can be performed for one row or multiple rows.

Select – a simple select or a complex one, that can be also saved in a table-valued function, or a view, or a stored procedure and returned from there.

Update – particular one on the fields of the table. NOT on the primary key. If the update is on the foreign key, first must be checked the existence of the new foreign key in the table in which is primary key. Update is performed for less records, maybe a group, but not very often. – preferable with WHERE condition. The update SHOULD NOT BE PERFORMED for the PRIMARY KEY. If in the intermediate table of the relation many to

many are only the foreign keys set it also to be primary key, nothing should be update (just a message will be ok).

Delete – particular one, for only some records or for all. The order of inserting is first the table in which is primary key, so that we can extract that primary key and set it to be foreign key, and only after the table with the foreign key. The order when we delete is the opposite one: first from the table that has the foreign key and then from the table that has the primary key. So, maybe when a delete from a table needs to be performed, other delete's will be need it to be performed (e.g. Delete from Shops may involve first the delete from ShopsTeas and only then the delete from Shops). – preferable with WHERE condition.

If consider proper, can be performed INSERT + SELECT + UPDATE (on the same record from insert – on a field) + SELECT (for double feedback) + DELETE (on the same record from insert and update) + SELECT (for double feedback).

Constraint(s) per table/column related to the validation of the data, can refer to primary key, foreign key, or other constraints.

PRIMARY KEY + FOREIGN KEY – can be threatad as follows:

- If are given as parameters – MUST be VALIDATED first
- If are not given as parameters – MUST be EXTRACTED from the corresponding tables.

PRIMARY KEY:

- If it is IDENTITY – nothing to be done – inserted automatically
- If it is NOT IDENTITY –
 - o If it is given as PARAMETER – MUST be VALIDATED – TO NOT EXIST IN THAT TABLE (for example, a count can be performed for that primary key given as parameter and the count to be 0)
 - o If it is NOT given as a parameter – the MAXIMUM value should be extracted and increased with 1, to get to a new value for the primary key (it should work also for the varchar type)

FOREIGN KEY:

- If it is given as PARAMETER – MUST be VALIDATED – TO EXIST IN THE TABLE IN WHICH IS PRIMARY KEY (for example, a count can be performed for that foreign key given as parameter in the table in which is primary key, and we check to be found there)
- If it is NOT given as a parameter – should be extracted from the table in which is primary key (the first value – top 1, minimum value, maximum value, average value, ..) and put it on the foreign key position – a variable can be used for this operation.

Validation:

IF validation is performed with success THEN INSERT ...
ELSE PRINT 'Cannot be performed due to the validation'

Stored procedure with a function used to validate the parameters:

<pre>USE Example_Lab1 GO CREATE PROCEDURE [dbo].[CRUD_Shops] @table_name Varchar(50), @name varchar(50), @city varchar(50), @nos int, @noc int, @noOfRows int AS BEGIN SET NOCOUNT ON; -- verify the parameters - at least one from the list IF (dbo.TestPrice(@nos)=1 and dbo.TestPrice(@noc)=1)</pre>	<pre>-- execute EXEC CRUD_Shops 'Shops', 'New Shop', 'Brasov', 2, 3, 10</pre>
--	---

```

BEGIN
-- CREATE=INSERT
insert into Shops(Name, City, NoOfShops, NoOfClients)
Values(@name, @city, @nos, @noc)
-- READ=SELECT
select * from Shops
-- UPDATE
update Shops set City='Cluj-Napoca'
where NoOfClients>10 and Name LIKE 'A%'
-- DELETE
delete from Shops
where NoOfClients=0
print 'CRUD operations for table ' + @table_name
END
ELSE
BEGIN
PRINT 'Error'
RETURN
END
END
END

```

Results		Messages			
	Pid	Name	City	NoOfShops	NoOfClients
1	1	New Shop	Brasov	2	3
2	2	New Shop	Brasov	2	3
3	3	New Shop	Brasov	2	3
4	4	New Shop	Brasov	2	3
5	5	New Shop	Brasov	2	3
6	6	New Shop	Brasov	2	3
7	7	New Shop	Brasov	2	3
8	8	New Shop	Brasov	2	3
9	9	New Shop	Brasov	2	3
10	10	New Shop	Brasov	2	3

Query executed successfully. DESKTOP-ATIN!

Stored procedure list_list_of_Parameters

Begin

Validation ok – INSERT + SELECT + UPDATE + DELETE

Validation NOT ok – message ‘Not ok – cannot be performed’

End

OR

Stored procedure list_list_of_Parameters

Begin

Validation ok – INSERT

Validation NOT ok – message_Insert ‘Not ok – cannot be performed’

SELECT

UPDATE

DELETE

End

ORGANIZE THE STRUCTURE OF THE CRUD OPERATIONS:

- Separate stored procedures for INSERT per each table, SELECT per each table, UPDATE per each table, DELETE per each table, AND a “union” procedure to execute CRUD per each table (15 stored procedures).

Also, please, prepare EXECUTION scenarios – with success and without success for each table.

Pentru fiecare tabel implicat în relația **M:N**, creați câte un trigger DML pentru operațiile de *update* și *delete*. Se va crea câte un tabel de *logging* pentru fiecare tabel implicat, care să conțină informațiile complete ale înregistrărilor actualizate sau șterse, tipul operației efectuate (*update* sau *delete*), data și ora operației și numele *login*-ului SQL care a efectuat operația. În cazul operației de *update*, trigger- ul va insera în tabelul de *logging* înregistrările actualizate în forma lor originală (versiunea de dinainte de *update*). În cazul operației de *delete*, trigger-ul va insera în tabelul de *logging* înregistrările șterse.

A trigger can be defined on one operation or multiples.

Create table Product(Pid int primary key identity, Name varchar(50), OperationDate date, Quantity int)	/* Log any operations performed on Product table */ create table Log_table(Lid int primary key identity, TableName varchar(50), OperationType varchar(50), AffectedRows int, ExecutionDate datetime)
---	---

```
insert into Product Values ('tomatoes', '2018-11-11', 5), ('potatoes', '2018-12-11', 6)
delete from Product Where Quantity=6 and Name LIKE 'p%'
update Product set Quantity=Quantity-1 where Name LIKE '%es'
```

	Lid	TableName	OperationType	AffectedRows	ExecutionDate
1	2	Product	I	0	2025-12-02 11:49:30.247
2	3	Product	D	0	2025-12-02 11:49:38.047
3	4	Product	U	0	2025-12-02 11:49:57.707

Here, we can have (informațiile complete ale înregistrărilor actualizate sau șterse, tipul operației efectuate (*update* sau *delete*), data și ora operației și numele *login*-ului SQL care a efectuat operația.)

<pre> /* Generic trigger for INSERT / DELETE / UPDATE */ create table Logging(Lid int primary key identity, TableName varchar(50), TableAttributes ... /* all table attributes */ OperationType varchar(50) check (OperationType IN ('INSERT', 'UPDATE', 'DELETE')), AffectedRows int, ExecutionDate datetime, sLogin nvarchar(128)) </pre>	<pre> -- @@SPID - the session id for the current user process SELECT @@SPID AS 'ID', SYSTEM_USER AS 'Login Name', USER AS 'User Name' SUSER_SNAME() -- Is the optional login security identification number - nvarchar(128) </pre>
--	---

Examples on **Product** table:

<pre> /* Logging table for DELETE */ create table LoggingDelete_Product(Lid int primary key identity, TableName varchar(50), /* all table attributes */ LPid int, LName varchar(50), LOperationDate date, LQuantity int OperationType varchar(50) check (OperationType IN ('INSERT', 'UPDATE', 'DELETE')), AffectedRows int, ExecutionDate datetime, sLogin nvarchar(128)) </pre>	În cazul operației de <i>delete</i>, trigger-ul va insera în tabelul de <i>logging</i> înregistrările șterse. <pre> /* Trigger for DELETE */ CREATE TRIGGER Delete_Product ON Product FOR DELETE AS BEGIN INSERT INTO LoggingDelete_Product(TableName, LPid, LName, LOperationDate, LQuantity, OperationType, AffectedRows, ExecutionDate, sLogin) SELECT 'Product', deleted.Pid, deleted.Name, deleted.OperationDate, deleted.Quantity, 'Delete', @@ROWCOUNT, GETDATE(), SUSER_SNAME() FROM deleted END GO </pre>
<pre> /* Logging table for UPDATE */ create table LoggingDelete_Product(Lid int primary key identity, TableName varchar(50), /* all table attributes */ LPid int, LName varchar(50), LOperationDate date, LQuantity int OperationType varchar(50) check (OperationType IN ('INSERT', 'UPDATE', 'DELETE')), AffectedRows int, ExecutionDate datetime, sLogin nvarchar(128)) </pre>	În cazul operației de <i>update</i>, trigger- ul va insera în tabelul de <i>logging</i> înregistrările actualizate în forma lor originală (versiunea de dinainte de <i>update</i>). <pre> /* Trigger for UPDATE */ CREATE TRIGGER Update_Product ON Product FOR UPDATE AS BEGIN INSERT INTO LoggingDelete_Product(TableName, LPid, LName, LOperationDate, LQuantity, OperationType, AffectedRows, ExecutionDate, sLogin) SELECT 'Product', deleted.Pid, deleted.Name, deleted.OperationDate, deleted.Quantity, 'Update', @@ROWCOUNT, GETDATE(), SUSER_SNAME() FROM deleted END GO </pre>
OR, only one trigger for both UPDATE and DELETE operations	

<pre> /* Logging table for UPDATE & DELETE */ create table Logging_Product(Lid int primary key identity, TableName varchar(50), /* all table attributes */ LPid int, LName varchar(50), LOperationDate date, LQuantity int OperationType varchar(50) check (OperationType IN ('INSERT', 'UPDATE', 'DELETE')), AffectedRows int, ExecutionDate datetime, sLogin nvarchar(128)) </pre>	<pre> /* Trigger for UPDATE & DELETE */ CREATE TRIGGER Update_Product ON Product FOR UPDATE, DELETE AS BEGIN DECLARE @operation CHAR(1) SET @operation = CASE WHEN EXISTS(SELECT * FROM inserted) AND EXISTS(SELECT * FROM deleted) THEN 'UPDATE' WHEN EXISTS(SELECT * FROM inserted) THEN 'INSERT' WHEN EXISTS(SELECT * FROM deleted) THEN 'DELETE' END IF @operation IS NULL RETURN INSERT INTO Logging_Product(TableName, LPid, LName, LOperationDate, LQuantity, OperationType, AffectedRows, ExecutionDate, sLogin) SELECT 'Product', deleted.Pid, deleted.Name, deleted.OperationDate, deleted.Quantity, @operation, @@ROWCOUNT, GETDATE(), SUSER_SNAME() FROM deleted END GO </pre>
* try also inserted table in the trigger ☺ (instead of deleted – for update) – - deleted = the removed value - inserted = the new value	
<pre> /* Execution with the trigger for DELETE */ delete from Product Where Quantity=6 and Name LIKE 'p%' </pre>	<pre> /* Execution with the trigger for UPDATE */ update Product set Quantity=Quantity-1 Where Quantity=6 and Name LIKE 'p%' </pre>

De asemenea, creați cel puțin 2 *view*-uri peste tabelele selectate pentru operațiile CRUD. Pentru tabelele folosite în *view*, creați indecși *nonclustered*. Pentru a vă asigura că indecșii pe care i-ați creat sunt utili, puteți verifica utilizarea acestora cu *Dynamic Management Views* sau prin afișarea planului de execuție al interogării. Pentru a afișa planul de execuție al interogării, activați butonul **Include Actual Execution Plan** din **SQL Server Management Studio (SSMS)**. În cazul în care constatați că indecșii nu sunt utilizați, este necesar să reconsiderați alegerea acestora sau să populați tabelele cu mai multe date.

Views must be created on the tables used for the CRUD operations and be relevant and also to include ALL the tables used in the stored procedures (for example, view_1 uses 2 tables and view_2 uses 3 tables, so, 5 tables).

Non-Clustered Indexes

USE Example_Lab1

GO

-- Find an existing index named Nix_Name and delete it if found.

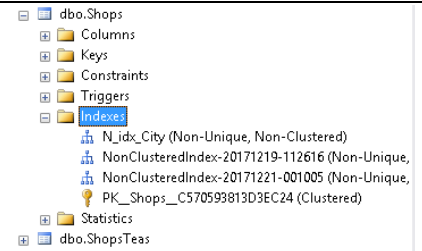
IF EXISTS (SELECT name FROM sys.indexes WHERE name = N'N_idx_City')

DROP INDEX N_idx_City ON Shops;

GO

-- Create a nonclustered index called N_idx_City on the Shops table using the City column.

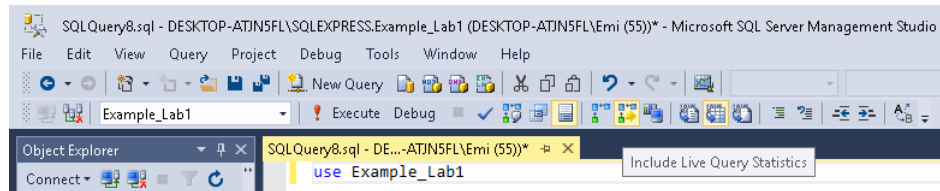
CREATE NONCLUSTERED INDEX N_idx_City ON Shops (City); GO



Check Clustered / Non-Clustered Indexes (Instead of Dynamic Management Views and Functions)

Check the indexes – check **Include Live Query Statistics** - when run a query.

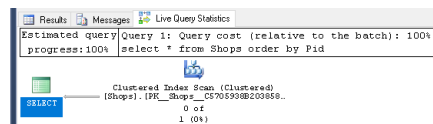
After an update / order by / ..., the order of the records is changed. For example, the indexes become 'un-ordered' (1, 2, 3, ... -> 3, 1, 2, ...). To choose the 'best' index, verify with the menu Include Live Query Statistics.



-- check Include Live Query Statistics

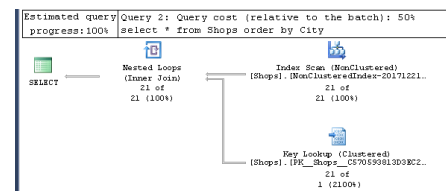
-- on Pid (the primary key) there is a clustered index

select * from Shops
order by Pid



-- on City I have a nonclustered index

select * from Shops order by City



By moving the mouse through the indexes, one can check the properties...

Index Scan (NonClustered)	
Scan a nonclustered index, entirely or only a range.	
Estimated operator progress: 100%	
Physical Operation	Index Scan
Logical Operation	Index Scan
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Number of Rows Read	21
Actual Number of Rows	21
Actual Number of Batches	0
Estimated I/O Cost	0.003125
Estimated Operator Cost	0.0033051 (34%)
Estimated Subtree Cost	0.0033051
Estimated CPU Cost	0.0001801
Estimated Number of Executions	1
Number of Executions	1
Estimated Number of Rows	21
Estimated Row Size	40 B
Actual Rewinds	0
Ordered	True
Node ID	1
Object	
[Example_Lab1].[dbo].[Shops].[NonClusteredIndex-20171221-001005]	
Output List	
[Example_Lab1].[dbo].[Shops].[Name], [Example_Lab1].[dbo].[Shops].[NoOfShops], [Example_Lab1].[dbo].[Shops].[NoOfClients]	
Seek Predicates	
Seek Keys[1]: Prefix: [Example_Lab1].[dbo].[Shops].[Pid] = Scalar Operator([Example_Lab1].[dbo].[Shops].[Pid])	

Key Lookup (Clustered)	
Uses a supplied clustering key to lookup on a table that has a clustered index.	
Estimated operator progress: 100%	
Physical Operation	Key Lookup
Logical Operation	Key Lookup
Actual Execution Mode	Row
Estimated Execution Mode	Row
Storage	RowStore
Number of Rows Read	21
Actual Number of Rows	21
Actual Number of Batches	0
Estimated I/O Cost	0.0004451 (66%)
Estimated Operator Cost	0.0064451
Estimated Subtree Cost	0.0001581
Estimated CPU Cost	0.0001581
Estimated Number of Executions	21
Number of Executions	21
Estimated Number of Rows	48 B
Estimated Row Size	48 B
Actual Rewinds	0
Ordered	True
Node ID	3
Object	
[Example_Lab1].[dbo].[Shops].[PK_Shops_C570593813D3EC24]	
Output List	
[Example_Lab1].[dbo].[Shops].[Name], [Example_Lab1].[dbo].[Shops].[NoOfShops], [Example_Lab1].[dbo].[Shops].[NoOfClients]	
Seek Predicates	
Seek Keys[1]: Prefix: [Example_Lab1].[dbo].[Shops].[Pid] = Scalar Operator([Example_Lab1].[dbo].[Shops].[Pid])	

Check all indexes and some other properties (leaf number after Insert, update, delete)

```
SELECT  
OBJECT_NAME(A.[OBJECT_ID]) AS [OBJECT  
NAME],  
I.[NAME] AS [INDEX NAME],  
A.LEAF_INSERT_COUNT,  
A.LEAF_UPDATE_COUNT,  
A.LEAF_DELETE_COUNT  
FROM  
SYS.DM_DB_INDEX_OPERATIONAL_STATS  
(NULL,NULL,NULL,NULL) A INNER JOIN
```

OBJECT NAME	INDEX NAME	LEAF_INSERT_COUNT	LEAF_UPDATE_COUNT	LEAF_DELETE_COUNT
Shops	NonClusteredIndex-20171219-112616	45	0	0
Shops	PK_Shops_C570593813D3EC24	45	0	0
Shops	PK_Shops_C570593813D3EC24	11	0	0
Possibilities	PK_Possibilities_C4E5D78AE50538D	3	0	0
Ingredients	PK_Ingredients_C4E5D78AE50538D	3	0	0
Teas	PK_Teas_C1FFD861954FD8B9	200	2	0
ShopsTeas	pk_Teas	16800	0	11148
Managers	pk_ShopsManagers	0	0	0
Shops	NonClusteredIndex-20171219-112616	11	0	0
Interim	NULL	4200	0	0

```

SYS.INDEXES AS I ON I.[OBJECT_ID] =
A.[OBJECT_ID] AND I.INDEX_ID = A.INDEX_ID
WHERE
OBJECTPROPERTY(A.[OBJECT_ID], 'IsUserTable') =
1

```

Show index plan

```

SET NOCOUNT ON;GO
SET SHOWPLAN_ALL
ON;GO
SELECT City
FROM Shops
WHERE NoOfClients
BETWEEN 1 AND 5;
GO
SET SHOWPLAN_ALL
OFF;

```

StmtId	Text	StmtId	NodeId	Parent	PhysicalOp	LogicalOp
1	SELECT City FROM Shops WHERE NoOfClients BETWEEN 1 AND 5;	1	1	0	NULL	NULL
2	Clustered Index Scan(OBJECT:([Example_Lab1].[dbo].[Shops].[PK_Sho...])	1	2	1	Clustered Index Scan	Clustered Index Scan

Argument	DefinedValues	EstimateRows
1	NULL	21
OBJECT:([Example_Lab1].[dbo].[Shops].[PK_Sho...]	[Example_Lab1].[dbo].[Shops].[City]	21

EstimateID	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecutions
NULL	NULL	NULL	0.0033051	NULL	NULL	SELECT	0	NULL
0.003125	0.0001801	40	0.0033051	[Example_Lab1].[dbo].[Shops].[City]	NULL	PLAN_ROW	0	1